

Automatic Generation of Submodel-Specific Classes with Predefined Meta-information Based on Submodel Templates

Igor Garmaev, Torben Miny, Tobias Kleinert
Chair of Information and Automation Systems for Process and Material Technology
RWTH Aachen
Aachen, Germany
{i.garmaev, t.miny, kleinert}@iat.rwth-aachen.de

Abstract—Asset Administration Shells are an essential component of Industry 4.0, and submodels are crucial to their functioning. However, the current method of generating submodels involves writing custom code, which can lead to code duplication and inconsistencies across submodels. To address these challenges, we propose an automatic generation of submodel-specific classes with predefined meta-information based on submodel templates. Our proposed solution aims to reduce code duplication, provide support in integrated development environments, ensure consistency across submodels, and facilitate the immediate publishing of submodel software development kits. This paper discusses the current approaches for instantiating submodels and their problems, the proposed approach, its requirements, advantages, and an outlook.

Keywords—Asset Administration Shells, Submodels, Automatic Code Generation, Submodel Templates

I. INTRODUCTION

The Asset Administration Shell (AAS), a key component of Industry 4.0, has gained significant importance in recent years. The AAS serves as a standardized model for the digital representation of physical and virtual assets, enabling companies to seamlessly import and export data between different systems. In this context, submodels play a crucial role as a vital part of AAS by portraying specific aspects of assets, thus providing a flexible and expandable framework for data integration.

However, the integration of the AAS into existing systems presents a challenge to companies: creating multiple scripts to instantiate different submodels for each asset and subsequently populating these with data from current systems. These scripts demand considerable effort and the manual creation of extensive code can give rise to numerous issues. These include a high risk of errors, inconsistencies in the created submodels, potential code duplication across different companies, and challenges in maintaining the code.

In this paper we propose a solution to facilitate these issues: an implementation of a generator of submodel-specific classes based on submodel templates. Our solution, built on the Software Development Kit (SDK) BaSyx-Python-SDK, generates submodel-specific classes with filled meta-information derived from submodel templates. The resulted classes serve as child classes of BaSyx-Python-SDK classes, representing classes of the AAS Metamodel.

These classes can then be used in scripts to instantiate submodels and populate them with data. The use of these

classes not only simplifies the process, enabling shorter scripts for submodel instantiation, but also offers submodel-specific support within Integrated Development Environment (IDE), and ensures consistency between instantiated submodels and their templates.

The rest of the paper is structured as following: Section II gives an overview of the AAS concept, the BaSyx-Python-SDK and submodel templates. Section III describes current approaches to create submodel-instances and their advantages and disadvantages. Our implementation is presented in Section IV and validated in Section V. The paper closes with a conclusion and an outlook in Section VI.

II. ASSET ADMINISTRATION SHELL

A. General

The Asset Administration Shell (AAS) is a concept developed within the scope of Industry 4.0 and represents the virtual representation of an asset [1]. An asset is understood to be a physical or logical object that is managed by an organization and has value for it [2]. For the interoperable exchange of information about an asset, an AAS metamodel was developed in UML.

Fig. 1 shows an excerpt from the UML class diagram of the AAS metamodel. The main classes *AssetAdministrationShell* and *Submodel* together represent the virtual representation of an asset. While the *AssetAdministrationShell* class has administrative information about the asset (such as a globally unique identifier of the asset or the type of asset as an instance or type), objects of the *Submodel* class contain application-specific information about the asset. Each *Submodel* object, represents an aspect of the asset, such as nameplate, topology, or documentation information. A *Submodel* object has a collection of *SubmodelElement* objects to represent the information. Different subtypes can be used depending on the type of information to be represented. For example, the *Property* class is used to represent scalar values, while the *Range* class is used for value ranges, and the *SubmodelElementCollection* class used for a collection of *SubmodelElement* objects to build a logical data tree recursively. Other classes make it possible to create references or other structures. Both *Submodel* and *SubmodelElement* classes inherit from *Qualifiable*, giving them the possibility to be specified by one or more qualifiers. Each qualifier is essentially represented by a type-value pair. Depending on the used qualifier kind, it may provide additional statements regarding various aspects of the qualified element. For instance, it could provide details with respect to the value of the qualified element, the semantic definition (concept) of the qualified element, or even the

existence and other meta information related to the type of the qualified element [5].

B. BaSyx-Python-SDK

The BaSyx-Python-SDK (formerly known as PyI40AAS - Python Industry 4.0 Asset Administration Shell [3]) is an SDK developed in Python by the Chair of Information and Automation Systems for Process and Material Technology at RWTH Aachen [4]. The open-source software is an implementation of the technology-neutral UML model published by the Plattform Industrie 4.0 for representing information in the Asset Administration Shell and is available in [4]. It is compliant with the metamodel defined in [5] and [6], and provides Python classes for the AAS-Metamodel, as well as a comprehensive Python-based SDK for working with AAS.

It's worth noting that there are also other SDKs available for the Asset Administration Shell [3]. However, the BaSyx-Python-SDK was chosen as this paper was developed in the wake of the “Basys4Forestry” project, where the BaSyx-Python-SDK was primarily used as the SDK.

C. Submodel Template

The AAS metamodel provides numerous opportunities for modeling information in the context of Industry 4.0. To achieve better semantic interoperability, the concept of submodel templates has been introduced. According to [5], a submodel template is defined as a “specification of the common features of an object in sufficient detail that such an object can be instantiated using it.”

A submodel template defines the structure of submodels for a specific aspect or information model and assigns a fixed identifier to their semantics. These submodel templates enable the responsible parties of the AAS to create and provide standardized models that represent specific aspects of an asset [7].

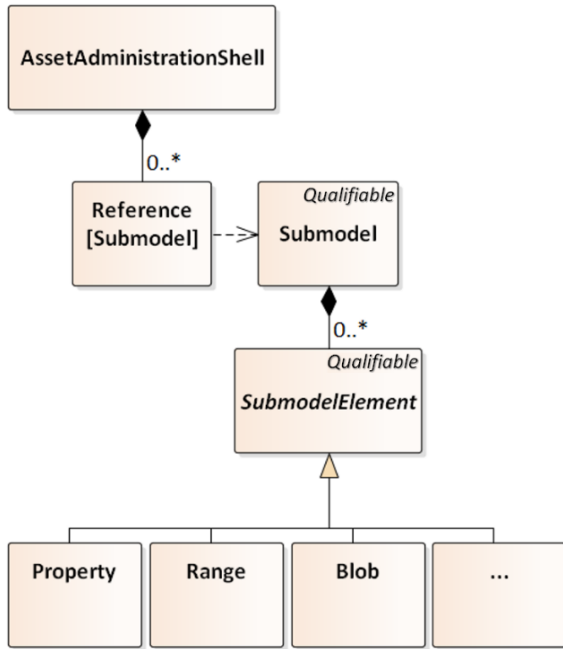


Fig. 1. Simplified UML class diagram of the Asset Administration Shell metamodel.

Within a submodel template, all intended objects, including their idShorts, descriptions, and identifiers of concept definitions (semanticIds), are defined. This ensures that during information exchange, it can be determined which information is contained in a compliant submodel object and how it can be identified and interpreted. Templates also define cardinalities, such as specifying whether an element is optional or mandatory. The only elements that cannot be defined within the template itself are the unique ID of the submodel instance and the property values [5].

The submodel template consists of computer-readable information, such as AASX package files (e.g., template, template with qualifiers, samples), as well as human-readable information in the form of a submodel template specification [7].

However, existing specifications do not explicitly describe the specific manner of instantiation and the dependency between a submodel instance and a submodel template. In [8], two fundamental variants were described regarding the interpretation of templates: whether they are copy templates or information model types. The latter was deemed more appropriate in [8], and in this paper, we offer a solution that aligns with this approach.

III. APPROACHES FOR CREATING SUBMODEL INSTANCES

According to the authors, the creation of AAS should primarily be done using scripts and programs. Therefore, the following approaches and further discussion focuses on the creation of submodel instances from the perspective of software development, i.e., the development of scripts and programs. It is assumed that the generation of submodels and AAS is done with the help of known SDKs.

As mentioned in Section II no specific manner of instantiation and the dependency between a submodel instance and a submodel template is directly described in specifications. According to existing definitions of submodel templates various approaches can be utilized for creating submodel instances, each with its own advantages and disadvantages. Below we will consider some of them.

A. Manually-Written Scripts/Functions

This approach represents manually written scripts or functions where all the predefined information of the submodel template, in addition to asset-specific values, is included. Fig. 2 provides a snippet of such a script for creating a *ManufacturerName*-Property of *DigitalNameplate* submodel [9], where the only asset-specific information is handed over in the parameter “value” and thus the most of the code is information from the submodel template.

```

manufacturerName = MultiLanguageProperty(
    value={'de': 'Muster AG'},
    id_short="ManufacturerName",
    semantic_id=Reference(
        key=(Key(type_=KeyElements.CONCEPT_DESCRIPTION,
            value="0173-1#02-AA0677#002",
            id_type=KeyType.IRDI,
            local=False),))

```

Fig. 2. Code-snippet for creating a “ManufacturerName” property defined in “DigitalNameplate” submodel template.

Although this approach allows immediate code writing with no need for using and/or developing any additional software other than one of the AAS SDKs, it suffers from several problems:

- Poor readability due to large scripts full of not asset-specific information, such as `idShort` or `semanticId` of the included elements from submodel template.
- High risk of oversights due to the manual copying of predefined information.
- Lack of submodel-specific support in integrated development environment.
- It requires a good knowledge of AAS and the used SDK.

B. Copy-and-Replace Scripts

These are scripts that read a submodel template, copy it and its submodel elements, and replace the example values with asset-specific values in the copied objects. This method allows for smaller scripts and immediate writing with no need for any software other than one of the AAS SDKs. However, it has some disadvantages:

- Risk of overlooked example values from submodel templates being included in the submodel instances.
- The used SDKs may not provide a copy functionality.
- Lack of submodel-specific IDE support.
- It requires a good knowledge of AAS and the used SDK.

C. Scripts Using Manually-Written Submodel-Specific Classes

This approach involves the creation of custom classes for a specific submodel and its elements, where all the predefined information is included. These classes then will be used in scripts to instantiate corresponding asset related Submodel/SubmodelElement instances. Fig. 3 shows an UML-diagram of the *DigitalNameplate*-submodel and Fig. 4 shows a snippet of a script for creating a *DigitalNameplate*-submodel, including mandatory SubmodelElements such as properties *ManufacturerName*, *URIOfTheProduct*, *ManufacturerProductDesignation*, *YearOfConstruction*, and the submodel element collection *ContactInformation* using submodel-specific class for instantiating of *DigitalNameplate*-submodel instances.

While this method improves readability, allows for shorter scripts to produce submodel instances, enables submodel-specific IDE support and facilitates scripts for instantiating submodels, it has the following disadvantages:

- It suffers from the potential for code duplication, as each company would write these for each submodel.
- It requires a significant amount of code for each submodel.
- High risk of oversights due to the manual copying of predefined information and too large codebase.

- The implementation of the classes requires a good knowledge of AAS and the used SDK.

D. Scripts Using Generated Submodel-Specific Classes

This method is similar to the previous one, but the classes are automatically generated based on the submodel template. Such it keeps all advantages of the method C but excludes/minimizes most of disadvantages. The problems of code-duplication and big codebase for submodel classes are not significant anymore, as classes are automatically generated based on the submodel template. It significantly reduces coding efforts and time, allows immediate creation of submodel-specific classes, and helps avoid oversights and inconsistencies due to machine-reading of predefined information and generating classes based on it. Moreover, no deep knowledge of AAS is required to generate the classes and use them if a submodel template is given. However, the implementation of the code generator does require a deep understanding of AAS and the used SDK.

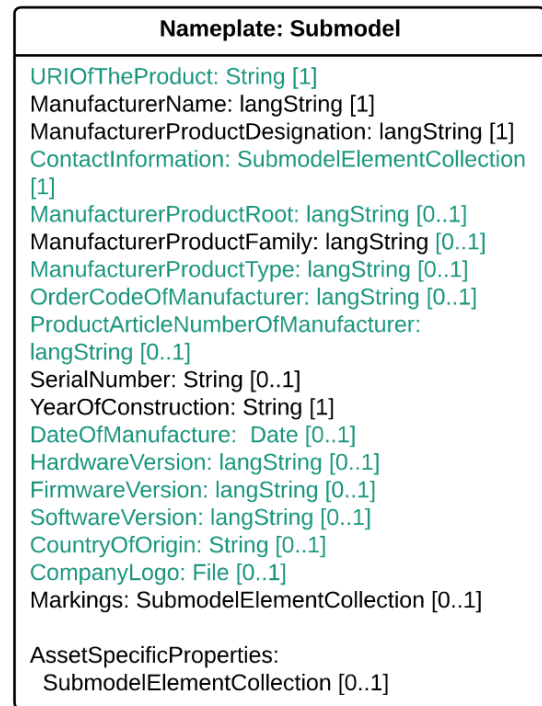


Fig. 3. UML-Diagram for *DigitalNameplate*-submodel [9]

```
nameplate = Nameplate(  
    identification=Identifier(  
        id='www.example.com/ids/sm/1225_9020_5022_1974',  
        id_type=IdentifierType.IRI),  
    uRIOfTheProduct='https://www.example.com/1234/5678',  
    manufacturerName={'de': 'Muster AG'},  
    manufacturerProductDesignation={'en': 'ABC-123'},  
    yearOfConstruction="2022",  
    contactInformation=Nameplate.ContactInformation(  
        nationalCode={'en': 'DE'},  
        cityTown={'de': 'Musterstadt'},  
        street={'Musterstrasse 1'},  
        zipcode={'de': '12345'})  
)
```

Fig. 4. Code-snippet for creating an instance of *DigitalNameplate*-submodel with specialized submodel-specific classes.

After considering the benefits and drawbacks of each approach, we decided to implement the generator needed for the last approach due to superior advantages of the corresponding approach.

At present, the authors are not aware of any alternatives to the mentioned generator functionalities, except the Generic Forms Preset concept found in the AASX Package Explorer [10]. This concept relies on the generation of presets based on computer-readable submodel templates, and it is utilized to render a visual plugin. Users can then complete information about submodel elements in the plugin and instantiate a submodel [7]. This solution is designed with a graphical user interface in mind, where users manually create submodel instances. However, it is not intended for coding purposes.

IV. IMPLEMENTATION OF THE GENERATOR

Our proposed solution involves the automatic generation of corresponding Submodel/SubmodelElement classes with predefined meta-information based on submodel templates to reduce the risk of errors, minimize development time and effort, and simplify the process of creating submodel instances. The implementation was realized on the base of BaSyx-Python-SDK for two versions of it, which supports versions V2.0.1 and V3.0 of the AAS-Metamodel. The source code for our implementation can be found in [13].

The general workflow of the submodel class generation is shown in Fig. 5. The implemented generator takes as input the submodel template and deserializes its elements into python-objects with BaSyx-Python-SDK. Then it iterates over all included elements in submodel template and creates for each of them a class based on the information saved in the element and predefined Class-Code-Template for this type of element. The generated class is a child-class of BaSyx-Python-SDK class into which the element of submodel template was deserialized. The resulted code with definition of generated classes will be saved in a python file. The user then can use the generated classes to initialize Instances of included SubmodelElements of the submodel and submodel itself and use all functionalities provided by BaSyx-Python-SDK to handle the submodel instances.

To describe the detailed generation process we will describe each of four stages depicted in Fig.5. To illustrate this process, we will utilize the example of class generation for the *DigitalNameplate*-submodel template, which content is shown in UML-diagram in Fig. 3.

A. Input

Firstly, the file with the submodel template (XML, JSON or AASX) is read and deserialized in python-objects of appropriate classes of BaSyx-Python-SDK, which represent classes of the AAS Metamodel. Here the submodel and included submodel elements saved in the file are deserialized into the object instances of classes from BaSyx-Python-SDK representing classes of AAS-Metamodel: a submodel Nameplate to a class Submodel, the property ManufacturerName to a class Property, a submodel element collection Address to SubmodelElementCollection, etc.

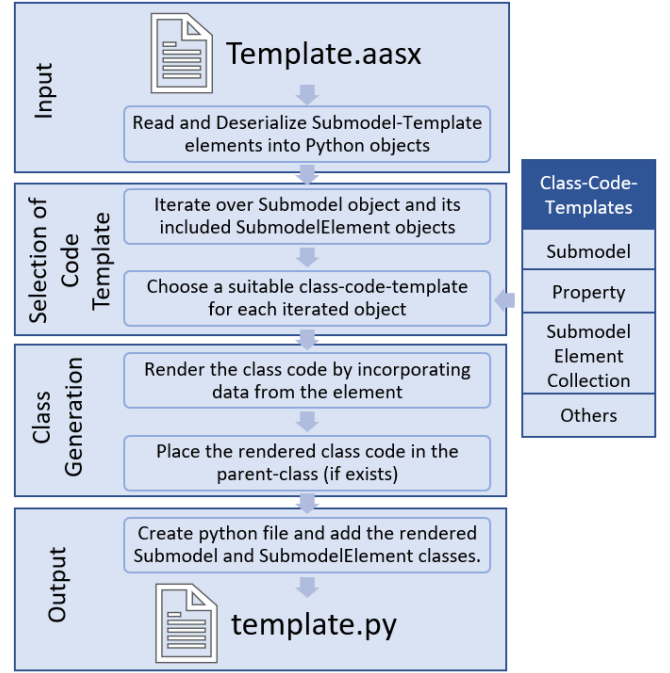


Fig. 5. Stages and steps of submodel-specific class generation workflow.

B. Selection of Code-Template

Once the submodel template has been read, the generator begins the iteration through the submodel and included submodel elements. For a submodel and each submodel element an appropriate pre-defined class-code-template according to the type of the current iterated object will be chosen based on defined type-template mapping table.

Such code-templates are given for *Submodel*-Type and each of *SubmodelElement*-Types such as *Property* or *SubmodelElementCollection*. They define the structure and logic of the resulting class and contain variables that function as placeholders within the template. These variables will be substituted with information from the currently iterated object, enabling the generation and rendering of a class.

All these templates share a foundation in the generic class template, a snippet of which is shown in Fig. 6. This template sets out the broad structure of the classes generated, consisting of blocks that outline basic components such as an argument list for the initialization function. Other templates extend this base template by replacing entire blocks and defining their own structure and logic.

Code-templates typically require the following information to render/generate code for submodel- or SubmodelElement-specific classes:

- The name of the custom submodel or SubmodelElement class to be generated (e.g., "Nameplate" or "ManufacturerName").
- The Name of the parent BaSyx-Python-SDK class (e.g., "Submodel" or "Property").
- Names of arguments and, if applicable, default values of arguments for the initialization function of the generated class.
- Types/typehints of initialization function arguments.


```

{% block class_def %}
class {{ cls_name }}({{ parent_cls_name }}):
{% endblock %}
    {% filter indent(width=4) %}
    {% block before_init %}
    {{ before_init_content }}
    {% endblock %}
    {% endfilter %}
    def __init__(
        self,
        {% filter indent(width=8) %}
        {% block init_args %}
        {% for arg in args %}
        {{ arg }}: {{ typehints.get(arg, 'Any') }},
        {% endfor %}
        {% endblock init_args %}
        {% block init_kwargs %}
        {% for key, value in kwargs.items() %}
        {{ key }}: {{ typehints.get(key, 'Any') }}={{ value }},
        {% endfor %}
        {% endblock init_kwargs %}
    {% endfilter %}
    ):
    ...

```

Fig. 6. Snippet of a class-code-template for a generic class.

If provided pre-defined templates do not fulfill requirements of the user, the user can define its own templates and hand it over to the generator. For instance, a user might want to implement specific constraints within the initialization function of the generated classes. An example of such a constraint could be ensuring that a German value is always provided when instantiating a `MultiLanguageProperty`.

C. Class Generation

In this section, we elaborate on the process of individual class generation based on the chosen Template and values stored in the currently iterated `SubmodelElement`. The generated `ManufacturerName` property class, as an illustrative example, is demonstrated in Fig. 7.

a) Core Generation Methodology

A generated class is a child class of the BaSyx-Python-SDK class of current `SubmodelElement`-Object, in case of `ManufacturerName` it is `MultiLanguageProperty`. The only method the generated class has is an `init`-function, which will be used for instantiating an Object, all other methods are inherited from the parent BaSyx-Python-SDK class, thus we can use all functionalities provided by the SDK for Submodels/SubmodelElements. The generated `init`-function has the same parameters as the `init`-function of parent class, however every parameter gets if given as default value the corresponding value of the read `SubmodelElement` from template, for example in the case of `ManufacturerName` as shown in Fig. 7 “`idShort`” is set per default to “`ManufacturerName`” and “`semanticId`” is set to `[IRDI] 0173-1#02-AA0677#002`. Thus, the user is not obligated to provide these metadata for instantiating a certain `SubmodelElement` of a standardized submodel if using the custom class, but still is able to replace these data with its own if desired.

```

class ManufacturerName(MultiLanguageProperty):
    def __init__(
        self,
        value: LangStringSet,
        id_short: str = "ManufacturerName",
        value_id: Optional[Reference] = None,
        category: Optional[str] = None,
        description: Optional[Dict[str, str]] = None,
        semantic_id: Optional[Reference] = Reference(
            key=(Key(
                type_=KeyElements.CONCEPT_DESCRIPTION,
                local=True,
                value="0173-1#02-AA0677#002",
                id_type=KeyType.IRDI,)),
            qualifier: Optional[Set[Constraint]] = None,
            kind: ModelingKind = ModelingKind.INSTANCE,
        ):
        ...

```

Fig. 7. Code-snippet of generated `ManufacturerName`-class to instantiate `ManufacturerName`-properties.

b) Handling of Classes Containing Other SubmodelElements

In the case of submodel or if the currently iterated `SubmodelElement` contains other submodel elements, e.g., submodel element collection `Address`, the generator iterates over the included submodel elements and creates for them classes with the same process as described above and put it inside the generated class. In case of submodel element collection `Address` the generator firstly creates classes of embedded submodel elements of the collection, such as `Street`, `CountryCode` and `CityTown`, and places it inside the class `Address`. This way it is guaranteed, that there are no classes with identical names inside one namespace, as, e.g., multiple submodel element collections can have a `SubmodelElement` with `idShort` “`Name`” which will be used for the created Class.

In case of `Submodel/SubmodelElementCollection` the `init`-function of the generated class doesn’t have a parameter `value`, where all included `SubmodelElements` can be handed over while instantiating, instead parameters are added, which represent each included `SubmodelElement`. This way the user gets an advice which specific `SubmodelElements` should be included for instantiation of the generated `Submodel/SubmodelElementCollection`-classes. For example, the `init`-function of the class of `SubmodelElementCollection` “`ContactInformation`” gets parameters “`nationalCode`”, “`cityTown`”, “`street`”, “`zipcode`” and others (See Fig. 4) where the user can hand over the corresponding submodel elements/values.

c) Handling of Qualifiers

During the implementation of the generator the current submodel templates provided by Industrial Digital Twin Association (IDTA) were reviewed to find out which qualifiers will be used by template creators. The only qualifier used in most of templates was *Multiplicity/Cardinality*-qualifier, which defines if a certain `SubmodelElement` is optional or mandatory or if multiple `SubmodelElements` can exist in the actual collection (e.g., `Submodel` or `SubmodelElementCollection`) [7].

If given, the generator uses this kind of qualifier to refine the init-functions of *SubmodelElement*-classes which may include other submodel elements by setting parameters which represent included SubmodelElements as optional or mandatory. For example, for the embedded submodel elements of the collection *Address* these qualifiers are given, which state that, e.g., *Street* property is mandatory and *Department* property is optional in the *Address* collection. So, the generator takes these qualifiers into account and constructs the init-function for *Address* collection class such, that the user must hand over values for mandatory embedded properties such as *Street* and is allowed to ignore parameters which correspond to optional ones.

D. Output

After iterations through the given submodel template and its submodel elements and creating a code with classes definition for each element the generator creates a python file where all generated classes for each specific submodel element included in the template will be placed. The user then can use classes defined in the file to instantiate a submodel instances without handing over already given meta-information in the submodel template, which significantly reduces coding efforts, time and the risk of oversights. The user profits from IDE help, that will inform, e.g., which elements the submodel or SubmodelElementCollection should include, which of these elements are mandatory and which are optional. That simplifies the process of creating submodel instances by providing some sort of documentation about the desired submodel directly into code. At the end the software developer, who will create scripts for instantiating submodel instances with the help of generated classes will create the instances much faster, with less inconsistencies and errors and will not have to deeply understand concepts of AAS and submodels particularly. For the developer process will look more like a normal object-oriented-programming, where the developer should pass only asset-related values to instantiate a submodel instance.

V. VALIDATION

We have successfully generated classes on base of *DigitalNameplate*[9] and *ContactInformation*[11] submodel templates of AAS-Metamodel V2.0.1 provided by IDTA, the source code of the generated classes can be found in [13]. We also generated classes based on our five own submodel templates of AAS-Metamodel V3.0 defined, which will be used by project partners in the “Basys4Forestry” project.

```
def create_holzliste(hl_json: dict, id_: int):
    polter_list = [create_polter(p) for p in hl_json['polter_list']]

    holzliste_sm = Holzliste(
        id_=f"https://forestmanager.de/aas/holzliste/{id_}",
        kopfdaten=create_kopfdaten(hl_json["headers"]),
        sortimentstyp=hl_json['sortiment'],
        sorte=hl_json['holzsorte'],
        holzart=hl_json['holzart'],
        polterListe=Holzliste.PolterListe(polter_list))
    return holzliste_sm
```

An example of utilization of our implementation is the function showcased in Fig. 8. The function uses the “Holzliste” (Engl.: timber list) class, generated from the corresponding submodel template, to instantiate a submodel instance. This function accepts a JSON representation of timber list, imported from our project’s partner internal data system. This JSON data is then used to instantiate a submodel instance. As depicted in the figure, the specifics of the internal submodel structure and predefined metadata from the used submodel template are encapsulated within the generated “Holzliste” class, greatly simplifying the process for the developer. This gives the impression of interacting with a standard Python data class. Additionally, IDE support is provided, showing the developer the required arguments for “Holzliste”, their expected types, and whether they are optional (if a default value is provided). The instantiation of submodel instances is essentially similar to the usual instantiation of typical Python data classes.

We successfully use the classes in the project to instantiate submodel-instances. Developers in the project who write software for mapping data from current systems to AAS and submodels particularly thanks to the generated classes do not have to deal with concepts of AAS which are new for them but can be concentrated on the mapping only.

The generator was validated for submodel templates which structure is strict and clear. The templates, where some parts are arbitrary, such as *TechnicalData*[12] where various not in template defined submodel elements can be added, were not validated and may be validated in the future works.

VI. CONCLUSIONS AND OUTLOOK

In this paper, we have shown different strategies for instantiating submodel instances, discussed their associated problems, and identified the most effective approach among them, which involves the instantiation of submodels using generated submodel-specific classes. Following this, we presented a software solution integral to our approach, that generates the necessary classes based on submodel templates. This approach, along with the accompanying software, was implemented and thoroughly tested during the “Basys4Forestry” project. By using submodel-specific classes to instantiate submodels and automating the process of generating these classes, our approach has effectively reduced developer-caused errors, minimized the time and effort required for submodel generation, and facilitated the implementation of scripts that create submodel instances.

```
model.generic_model.Holzliste
def __init__(self,
    id_: str,
    kopfdaten: Kopfdaten,
    sortimentstyp: str | Sortimentstyp,
    sorte: str | Sorte,
    holzart: str | Holzart,
    polterListe: PolterListe,
    preismatrix: Preismatrix | None = None,
    id_short: str = "Holzliste",
    display_name: LangStringSet | None = None,
```

Fig. 8. Function for submodel instantiation using the generated class ‘Holzliste’ and associated IDE support feature.

Currently, our solution builds upon the BaSyx-Python-SDK, thus limiting the code generator to Python alone. As various SDKs for AAS exist across multiple programming languages today, our proposed solution could offer considerable advantages if expanded to other languages. A potential workaround involves implementing a similar approach in “aas-core-meta”[14] and “aas-core-codegen”[15], libraries for generative and model-driven AAS SDK development[16]. Here “aas-core-meta” serves as an intermediate representation of the metamodel in Python, which could then be utilized by ‘aas-core-codegen’ to generate AAS SDKs for a variety of languages. The integration of our approach into these libraries would involve generating intermediate representation of submodel-specific classes within ‘aas-core-meta’, which would then be processed by ‘aas-core-codegen’, producing corresponding classes across various SDKs.

There are further opportunities to enhance our solution by incorporating additional types of qualifiers. For instance, we can explore the inclusion of checks for set values in the generated classes by utilizing qualifiers such as AllowedRange or AllowedValue. This would ensure that the generated classes adhere to specific constraints and provide more robust validation mechanisms.

Another area for future exploration is the compatibility of our generated classes with arbitrary submodel templates. Investigating how the generated classes can adapt and handle diverse submodel structures would improve the versatility and applicability of our approach.

To further advance the adoption and usability of our solution, we intend to generate submodel-specific classes for all submodel templates available in the IDTA library. By doing so, we aim to create a comprehensive software package that includes classes for all publicly published standardized submodels. This package will fasten the development process and enable easy integration of standardized submodels into various AAS implementations.

It is important to note that the success of our proposed solution relies on the availability of consistent submodel templates. Ensuring the standardization and reliability of these templates will contribute to the efficiency and effectiveness of the automated submodel generation process.

In summary, our approach of automatic generation of submodel-specific classes based on submodel templates has demonstrated significant advantages for submodel instantiation. With the future incorporation of additional qualifiers, compatibility with arbitrary submodel templates, and the creation of software packages with classes for all publicly available submodel templates, we anticipate a further reduction in development effort and increased adoption of standardized submodels and AAS in general in the industry.

ACKNOWLEDGMENT

This work is supported by German Federal Ministry of Education and Research in the scope of the “BaSys4Forestry” project with the funding reference number 01IS21074A.

- [1] W. Dorst, *Umsetzungsstrategie Industrie 4.0: Ergebnisbericht der Plattform Industrie 4.0*. Bitkom Research GmbH, 2015
- [2] DIN SPEC 91345: Reference Architecture Model Industrie 4.0 (RAMI4.0), Standard, DIN - Deutsches Institut für Normung, 2016
- [3] T. Miny, M. Thies, S. Heppner, I. Gamaev, L. Möller, and T. Kleinert, “Realisierung und Evaluation des Verwaltungsschalen-Metamodells,” *atp magazin*, vol. 63, no. 9. Vulkan-Verlag GmbH, pp. 60–67, Aug. 12, 2022. doi: 10.17560/atp.v63i9.2586.
- [4] Eclipse BaSyx Python SDK <https://github.com/eclipse-basyx/basyx-python-sdk>
- [5] Plattform Industrie 4.0, „Details of the Asset Administration Shell - Part 1 - The exchange of information between partners in the value chain of Industrie 4.0 (Version 3.0)“. Bundesministerium für Wirtschaft und Energie, Mai 2022.
- [6] Plattform I4.0, „Details of the Asset Administration Shell - Part 1: The exchange of information between partners in the value chain of Industrie 4.0 (Version 2.0.1)“, 2019. [Online]. Available: https://www.plattform-i40.de/PI40/Redaktion/EN/Downloads/Publikation/Details_of_the_Asset_Administration_Shell_Part1_V2.html.
- [7] Industrial Digital Twin Association, “GUIDELINE: HOW TO CREATE A SUBMODEL TEMPLATE SPECIFICATION”, 2022. [Online]. Available: <https://industrialdigitaltwin.org/wp-content/uploads/2022/12/I40-IDTA-WS-Process-How-to-write-a-SMT-FINAL-.pdf>
- [8] T. Miny, S. Heppner, I. Gamaev, T. Kleinert and B. Höper, “Asset Administration Shell Submodels – Copy Template or Information model type”, 24. Leitkongress der Mess- und Automatisierungstechnik AUTOMATION 2023, June 2023.
- [9] “IDTA 02006-2-0 Digital Nameplate for industrial equipment”, 20 October 2022, Industrial Digital Twin Association, [Online]. Available: https://industrialdigitaltwin.org/wp-content/uploads/2022/10/IDTA-02006-2-0_Submodel_Digital-Nameplate.pdf
- [10] AASX Package Explorer, <https://github.com/admin-shell-io/aasx-package-explorer>
- [11] “IDTA 02002-1-0 Submodel for Contact Information”, 24 May 2022, Industrial Digital Twin Association, [Online]. Available: https://industrialdigitaltwin.org/wp-content/uploads/2022/10/IDTA-02002-1-0_Submodel_ContactInformation.pdf
- [12] “IDTA 02003-1-2 Generic Frame for Technical Data for Industrial Equipment in Manufacturing”, 4 August 2022, Industrial Digital Twin Association, [Online]. Available: https://industrialdigitaltwin.org/wp-content/uploads/2022/10/IDTA-02003-1-2_Submodel_TechnicalData.pdf
- [13] aas-submodel-template-to-py <https://github.com/rwth-iat/aas-submodel-template-to-py>
- [14] aas-core-meta <https://github.com/aas-core-works/aas-core-meta>
- [15] aas-core-codegen <https://github.com/aas-core-works/aas-core-codegen>
- [16] N. Braunisch, M. Ristin-Kaufmann, R. Lehmann, and H. W. van de Venn, “Generative and Model-driven SDK development for the Industrie 4.0 Digital Twin,” 2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA). IEEE, Sep. 07, 2021. doi: 10.1109/etfa45728.2021.9613164.